

Track A Lab Guide

Gem Hunter: 9 Exercises in a Foreign Codebase

Prepared for Panteon Games — Codeo · Engineering + DevOps · 13:30 – 15:00

Today you work in the gem-hunter-workshop project — Unity's official match-3 sample. In this compressed single-trainer run, Ex01, Ex02, Ex06 and Ex07 are hands-on with verification from the terminal; Ex03–Ex05 and Ex09 are fast instructor demos, and Ex08 is briefed for you to complete after the workshop. Detailed steps and prompts live in each exercise's own file (Workshop/Ex0N_*.md); this guide is your roadmap for the day.

THE SYNC RULE — GIT CHECKPOINTS

Every exercise has a start branch (ex01-start ... ex09-start + complete), and each branch equals the finished state of the previous exercise. Anyone who falls behind or misses one syncs with a single command; your work is auto-stashed. The next branch is also the previous exercise's solution: `git diff ex06-start..ex07-start` = the canonical fix for BUG-312.

```
./catchup.sh ex05 — switch to ex05-start . ./catchup.sh list — all checkpoints
```

ENVIRONMENT CHECK (10 MIN)

- ☐ The gem-hunter-workshop clone is open; `unity status` shows the connected editor as "ready".
- ☐ With `git branch -a` you can see the ex01-start ... complete branches; you are currently on main.
- ☐ A claude session opens; you saw that the CLAUDE.md at the root is a stub for now — you write the real one in Ex02.
- ☐ You have played the game once: Level 1, the goals, the boosters.

THE NINE EXERCISES

1. Ex01 — Recon: map the architecture (13:40, hands-on)

The agent produces Workshop/ARCHITECTURE.md: scene flow, the 5 critical classes, match detection, the booster pattern, level data — every claim cites a file path. You verify at least two claims against the source.

2. Ex02 — Contract: the real CLAUDE.md (hands-on)

Turn the discovery into rules: protected areas, an address for every job, the verification ritual. Test: ask a fresh agent session "what are you not allowed to touch?" — it should answer from your contract.

3. Ex03 — LevelData: make Level 2 harder (instructor demo)

MaxMove 20»16, chest goal 6»11 — in scene data, through the CLI. Verify: in play mode, `eval 'return Match3.LevelData.Instance.MaxMove;'` and the HUD.

4. Ex04 — ComboCounter: use the existing hooks (instructor demo)

A ComboTracker subscribed to LevelData's OnGoalChanged/OnMoveHappened delegates. First question in plan review: is it inventing new plumbing, or using what exists?

5. Ex05 — Amethyst: the 7th gem (instructor demo)

A prefab copied from GemWhite, GemType 6, purple tint, registered into Level 1's Board.ExistingGems array through the CLI. Goal: zero C# changes. Verify: purple gems on the board + ExistingGems.Length = 7.

6. Ex06 — BUG-312: the root-cause hunt (14:30, hands-on)

The QA report: the move counter drops even on a bounced-back swap. The rule: no fix until the root cause is proven. The proof pair: RemainingMove before/after a failed swap — buggy 20»19, fixed 20»20.

```
$ unity command --project-path . eval 'return Match3.LevelData.Instance.RemainingMove;'
```

7. Ex07 — level_lint: your content validator (14:45, hands-on)

A CLI command that audits the open scene: LevelData/Board presence, MaxMove, goal integrity, whether goal gems are in the mix. Sabotage test: set a goal's Count to 0 » FAIL » revert.

8. Ex08 — FEAT-88: Cross Bomb, no template (briefing — solo after the workshop)

A bare ticket: a booster that clears row+column, following the SmallBomb pattern. You write the prompt; proof: on use, every gem in the row+column joins a single match (13+1).

9. Ex09 — Build: iOS from the terminal (14:55, instructor demo)

An async build from the live editor; while waiting, read from the source how the build preprocessor in LevelList.cs fills the scene list. If Build Settings drift from the LevelList asset the build stops itself once with "Level List had to be rebuilt, restart the build" and fixes the list — restart and it goes through. Bonus: why does it stop instead of continuing? The ~13 "More than one global light" errors in the report are pipeline-package noise; the result still says Succeeded.

```
$ unity command --project-path . build --target iOS --outputPath Builds/iOS --confirm true
$ unity command --project-path . build_status
```

TROUBLESHOOTING

Symptom	Fix
unity status is not "ready"	The editor may be compiling; wait. If it persists, check for Safe Mode with <code>unity pipeline list</code>
The agent heads for Settings/ or scene YAML	Reject the plan — the contract you wrote in Ex02 exists exactly for this
I fell behind / broke the scene	<code>./catchup.sh exNN</code> — your work is stashed and you are synced
I want to see the solution	<code>git diff exNN-start..exNN+1-start</code> — the canonical solution is always open
The editor is confused after a branch switch	<code>catchup.sh</code> refreshes the editor; if needed, reopen the scene via File > Open Recent

GETTING READY FOR THE CLOSING (16:40)

- ☐ Your BUG-312 proof pair is ready: the buggy 20»19 and fixed 20»20 eval outputs.
- ☐ Your `level_lint` command is in the `unity command` list and passed the sabotage test.
- ☐ The Cross Bomb briefing is in hand; the solo flight is planned for after the workshop.
- ☐ The build report (`build_status` output) is at hand; T1/T3 observations written into the metrics table.